

# Introduction to Databases

## Lecture 1: The "Why"

---

Amit Kumar Dhar

July 30, 2025

- **Lectures:** Do Attend! NOT Optional.
- **Labs:** Clarifies your concepts! Also NOT Optional
- **Quizzes:** Announced and Unannounced. 3 OR 4
- **Lab Evaluations:** Announced and Unannounced.
- **Mid-, End-Sem:** Highest weightage.

\* Subject to Change

\* May include a possibility of a project

# Evaluation

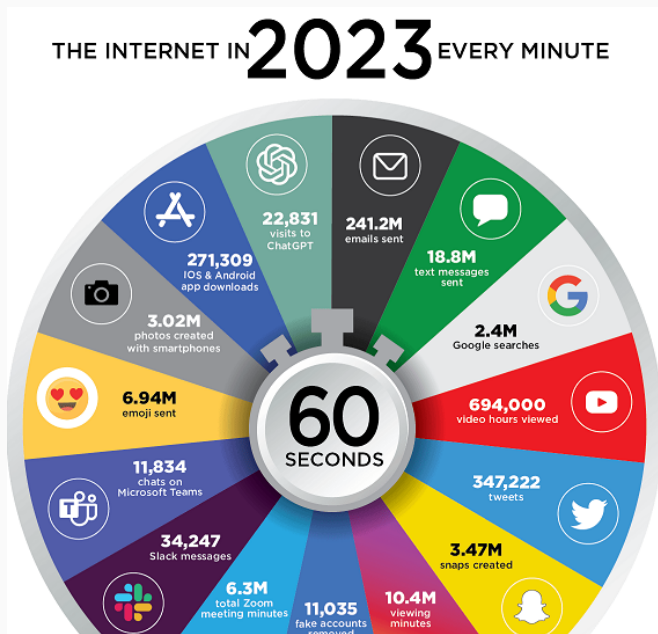
|                         |     |
|-------------------------|-----|
| <b>Mid-Sem:</b>         | 25% |
| <b>End-Sem:</b>         | 40% |
| <b>Quizzes:</b>         | 10% |
| <b>Lab Evaluations:</b> | 20% |
| <b>Attendance:</b>      | 5%  |

- Database Systems: The Complete Book, H Garcia-Molina, J Ullman, J Widom
- Database Management Systems, Ramakrishnan
- Fundamentals of Database Systems, Elmasri, Navathe

# Agenda

- **Motivation:** Why are we here? Why study databases?
- **The Core Problem:** What is a database and what problem does it solve?
- **The Old Way:** Why not just use files?
- **The DBMS Solution:** How a Database Management System saves the day.
- **Real-World Impact:** Examples from everyday life.

# Motivation: The Data Explosion



# Why Study Databases? The Job Market

Mastering databases opens doors to many high-demand careers:

- **Database Administrator (DBA):** Manages, secures, and ensures the performance of databases.
- **Data Engineer:** Builds the data pipelines and infrastructure to process data at scale.
- **Backend Developer:** Builds the server-side logic of applications, which almost always involves a database.
- **Data Analyst/Scientist:** Queries and analyzes data to extract insights and build predictive models.

**Bottom line:** These are well-paying jobs that are fundamental to the tech industry.

# Why Study Databases? Research Potential

Databases are a very active area of academic and industrial research:

- **Distributed & Cloud Databases:** How to build databases that can span thousands of machines across the globe (e.g., Google Spanner, Amazon Aurora).
- **AI/ML for Databases:** Can a database learn to optimize itself? (e.g., automatic indexing, query plan optimization).
- **New Data Models:** Designing databases for specialized data like graphs (social networks) or time-series (financial data).
- **Hardware-Conscious Databases:** Designing databases that take advantage of modern hardware like GPUs and persistent memory.



# What is Data?

- Data are raw, unorganized facts that need to be processed.
- For example, a customer's name, a product's price, a transaction's timestamp.
- These individual facts are called **data items**.

# What is a Database?

- A database is a **collection of related data**.
- It represents some aspect of the real world (a "miniworld").
- It is designed to be **logically coherent**, with some inherent meaning.

## **Example: University Database**

A collection of data about students, courses, and grades.

# What is a DBMS?

A **Database Management System (DBMS)** is a software package that allows us to create, manage, and query a database.

- It's the **software layer** that sits between the user and the physical data.
- Examples: MySQL, PostgreSQL, Oracle, SQL Server, MongoDB.

## Real-Life Example: Amazon

Think about all the data Amazon's DBMS manages:

- **Products:** Names, descriptions, prices, inventory levels.
- **Customers:** Usernames, passwords, shipping addresses, payment methods.
- **Orders:** Order IDs, items purchased, shipping status.
- **Reviews:** User ratings, comments.
- **And much more:** Seller information, Prime Video watch history, etc.

A DBMS makes this possible.

# The Core Question: Why Not Just Use Files?

- You could store your data in files (e.g., CSV, text files).
- This is called a "file-based system".
- Let's consider a simple bank application to see why this is a bad idea.

## Problem 1: Data Redundancy & Inconsistency

The same information may be duplicated in several files.

### **Bank Example**

A customer's address might be stored in a file for checking accounts and a separate file for savings accounts. If the customer moves, you must update both files. If you only update one, the data is now **inconsistent**.

## Real-Life Consequence of Inconsistency

- A bill is sent to the old address, causing customer dissatisfaction and late payment fees.
- A critical alert about a fraudulent transaction is missed.
- The bank might make a lending decision based on incomplete or incorrect data.

## Problem 2: Difficulty in Accessing Data

You have to write a new program to answer every new question.

### **Bank Example**

**Question:** "Find the total balance of all customers who live in California and have more than two accounts."

To answer this, you would need to write a program to:

1. Parse the checking account file.
2. Parse the savings account file.
3. Combine the data.
4. Filter by state.
5. Count the accounts for each customer.
6. Calculate the total balance.



## Problem 3: Data Isolation

Data is scattered in various files, and files may be in different formats (CSV, binary, etc.). This makes it very difficult to combine data from different sources.

## Problem 4: Integrity Problems

It is difficult to enforce constraints on the data.

### Bank Example

- An account balance should not be negative.
- An account type must be one of "Checking", "Savings", or "Loan".
- Every account must be associated with a valid customer.

In a file-based system, the application code is responsible for enforcing these rules. This is error-prone and can be bypassed.

## Problem 5: Atomicity Failures

It is difficult to ensure that a transaction is "all or nothing".

### **Bank Example: Fund Transfer**

A transfer of \$100 from account A to account B involves two steps:

1. Debit \$100 from A.
2. Credit \$100 to B.

What if the system crashes after step 1 but before step 2? \$100 has vanished! This operation must be **atomic**.

## Real-Life Consequence of Atomicity Failure

- The customer loses money.
- The bank's books are unbalanced, leading to a major accounting and legal issue.
- This is one of the most critical problems that a DBMS solves.

## Problem 6: Concurrent Access Anomalies

If multiple users are accessing and updating the same data, this can lead to inconsistencies. This is known as the "lost update" problem.

### **Bank Example**

Account A has \$500.

1. Teller 1 reads the balance (\$500).
2. Teller 2 reads the balance (\$500).
3. Teller 1 deposits \$100, and writes back \$600.
4. Teller 2 withdraws \$50, and writes back \$450.

The final balance is \$450. Teller 1's deposit has been lost!

## Problem 7: Security Problems

It is hard to provide different levels of access to different users.

### **Bank Example**

- A bank teller should be able to see a customer's balance, but not an employee's salary.
- A customer should only be able to see their own accounts.

With files, this is very difficult to enforce. You might have to rely on the operating system's file permissions, which are not granular enough.

# The DBMS to the Rescue!

A DBMS solves all of these problems by providing a set of powerful features.

## Feature 1: A Unified Data Model

A DBMS provides a way to model the data in a structured way (e.g., the relational model). This reduces redundancy and makes it easier to access the data.



## Feature 2: A High-Level Query Language

A DBMS provides a high-level language (like SQL) to query and manipulate data.

### **The Hard Question from Before**

"Find the total balance of all customers who live in California and have more than two accounts."

In SQL, this might be a single, declarative query. You specify **what** you want, not **how** to get it.

## Feature 3: Integrity Constraints

A DBMS allows you to define constraints on the data, which are enforced by the system.

### Example SQL Constraints

```
CREATE TABLE Accounts (  
    AccountNumber INT PRIMARY KEY,  
    Balance DECIMAL(10, 2) CHECK (Balance >= 0),  
    AccountType TEXT CHECK (AccountType IN ('Checking', 'Sa  
);
```

## Feature 4: Transactions & Atomicity (ACID)

A DBMS provides the concept of a **transaction**, which is a single logical unit of work. The DBMS guarantees that transactions are:

- **A**tomic: All or nothing.
- **C**onsistent: The database remains in a valid state.
- **I**solated: Concurrent transactions do not interfere with each other.
- **D**urable: Once a transaction is committed, it will survive failures.

## Feature 5: Concurrency Control

A DBMS uses sophisticated techniques (like locking) to ensure that concurrent transactions do not lead to anomalies like the "lost update" problem.

## Feature 6: Robust Security Model

A DBMS provides a security model that allows you to grant and revoke privileges to users and roles.

### Example SQL Security

```
-- A teller can see customer balances
```

```
GRANT SELECT ON CustomerAccounts TO teller_role;
```

```
-- But cannot see employee salaries
```

```
REVOKE SELECT ON EmployeeSalaries FROM teller_role;
```

# Amazon Example Revisited

How does a DBMS power Amazon?

- **Integrity:** Ensures that product prices are positive and order quantities are valid.
- **Transactions:** When you place an order, the DBMS ensures that your payment is processed, the inventory is updated, and your order is created **atomically**.
- **Concurrency:** Millions of users can browse and buy products at the same time without interfering with each other.
- **Security:** You can only see your own order history and personal information.

## Summary & Key Takeaways

- A DBMS is not just a data container; it's a complex software system that provides crucial services.
- Using a DBMS instead of a file-based system is essential for almost any real-world application.
- A DBMS provides data independence, efficient data access, data integrity and security, data administration, concurrent access, and crash recovery.

## Next Time...

- We will look at the different types of databases (Relational vs. Non-Relational).
- We will start to look at the relational model in more detail.
- We will begin our journey with SQL.